



POLITECHNIKA
LUBELSKA
WYDZIAŁ ELEKTROTECHNIKI
I INFORMATYKI

Praca dyplomowa inżynierska

na kierunku Informatyka
na specjalności Inżynieria oprogramowania

Integracja komponentu renderującego obiekty 3D z internetowym serwisem aukcyjnym

Integration of a component rendering 3D objects with an online auction service

Konrad Tomasz Nowak

99640

Maciej Ołdakowski

99648

Patryk Kamil Nowacki

99638

Promotor Dr Marcin Barszcz

Lublin 2025

Streszczenie

Komentarz: Streszczenie zostanie napisane po skończeniu całości pracy.

Abstract

Spis treści

1	Wstęp	3
2	Cel i zakres pracy	4
2.1	Cel pracy	4
2.2	Zakres pracy	4
2.3	Podział pracy	6
3	Analiza rynku	7
3.1	allegro.pl	7
3.2	amazon.pl	7
3.3	erli.pl	7
4	Użyte technologie i narzędzia	9
4.1	Element renderujący	9
4.1.1	Język C++	9
4.1.2	WebGPU	9
4.1.3	WebGPU-Cpp	9
4.1.4	WebAssembly	9
4.1.5	Emscripten	9
4.1.6	WGSL (WebGPU Shading Language)	10
4.1.7	CMake	10
4.1.8	Ninja	10
4.1.9	Biblioteki C++	10
4.2	System aukcyjny	11
5	Wprowadzenie do renderowania 3D	12
5.1	WebGPU	12
5.2	WebAssembly	13
5.3	Podsumowanie	13
6	Implementacja komponentu renderującego	14
6.1	Okno aplikacji	14
6.2	WebGPU	15
6.2.1	Adapter	16
6.2.2	Device	16
6.2.3	Command queue	16
6.2.4	Shader module	16
6.2.5	Depth texture	16
6.2.6	Bind group layout	16
6.2.7	Pipeline	17

6.2.8	Sampler	17
6.2.9	Textures	17
6.2.10	Geometry	17
6.2.11	Uniforms	17
6.2.12	Lighting uniforms	17
6.2.13	Bind group	17
6.3	Pętla główna	17

1 Wstęp

Z roku na rok rynek e-commerce dynamicznie się rozwija, przyciągając coraz większą liczbę użytkowników. Według badania „E-commerce w Polsce 2024” szacuje się, że w 2024 roku aż 75% internautów dokonuje zakupów w polskich sklepach internetowych, a 36% korzysta z zagranicznych platform e-commerce. Pandemia Covid-19 znacząco przyspieszyła ten trend — w 2020 roku wartość sprzedaży online w Polsce podwoiła się w porównaniu z rokiem poprzednim. Wraz z rozwojem rynku użytkownicy oczekują coraz bardziej innowacyjnych rozwiązań, które zwiększają ich zaufanie do zakupów online i poprawiają jakość doświadczeń zakupowych.

Jednym z kluczowych wyzwań w handlu internetowym, w szczególności w sektorze aukcji online, jest zapewnienie kupującym możliwości dokładnego zapoznania się ze stanem oferowanego przedmiotu. Tradycyjne platformy aukcyjne, takie jak `desa.pl`, `allegro.pl` czy `the-saleroom.com` opierają się głównie na zdjęciach i opisach, które nie zawsze w pełni oddają rzeczywisty stan produktu. Brak możliwości szczegółowego obejrzenia przedmiotu może prowadzić do nieporozumień i obniżać zaufanie kupujących. Rozwiązaniem tego problemu może być zastosowanie nowoczesnych technologii wizualizacji, które umożliwiają interaktywne przedstawienie produktów w trójwymiarowej przestrzeni.

W pracy inżynierskiej opisane zostanie wykorzystanie technologii WebGPU, wykorzystanej do przedstawienia trójwymiarowych obiektów. Obecnie najpowszechniejszym stosowanym standardem do renderowania grafiki 3D w przeglądarkach jest WebGL, na którym opiera się zdecydowana większość istniejących rozwiązań — od prostych wizualizacji produktów w sklepach internetowych po zaawansowane aplikacje typu konfigurator 3D czy przeglądarki modeli w branżach takich jak motoryzacja, meblarstwo i sztuka. Na WebGL bazują popularne biblioteki i silniki, m.in. `Three.js`, `Babylon.js`, `PlayCanvas` czy `Potree`, które znacznie upraszczają tworzenie złożonych scen 3D.

Aktualnie powstaje próba wprowadzenia nowego standardu WebGPU — następcy WebGL, który oferuje dostęp do nowoczesnych interfejsów programistycznych pozwalających na komunikację z procesorami graficznymi (Vulkan, Metal, Direct3D 12), znacznie wyższą wydajność, lepszą kontrolę nad pamięcią GPU oraz możliwość wykorzystywania compute shaders. W 2025 roku WebGPU jest już stabilnie wspierany w przeglądarkach opartych o silnik Chromium (Chrome, Edge, Opera) oraz w Firefoxie. Dzięki temu możliwe staje się osiągnięcie jakości i płynności renderowania dotychczas zarezerwowanej dla aplikacji natywnych, przy jednoczesnym zachowaniu pełnej kompatybilności z ekosystemem webowym. Wykorzystanie WebGPU otwiera nowe możliwości tworzenia fotorealistycznych i wysoce interaktywnych wizualizacji przedmiotów aukcyjnych bezpośrednio w przeglądarce, bez konieczności instalowania dodatkowego oprogramowania.

2 Cel i zakres pracy

2.1 Cel pracy

Celem pracy inżynierskiej jest zaprojektowanie i implementacja systemu informatycznego, składającego się z dwóch zintegrowanych części: komponentu renderującego modele trójwymiarowe oraz internetowego serwisu aukcyjnego. Praca zakłada przedstawienie implementacji obu elementów systemu oraz sposobu ich połączenia w jeden działający system informatyczny. Część odpowiedzialna za wizualizację modeli 3D będzie wykorzystywać zasoby procesora graficznego w celu zapewnienia płynnego i efektywnego renderowania. Przekompilowany do formatu WebAssembly kod komponentu graficznego zostanie poprawnie uruchomiony przez nowoczesne silniki przeglądarek, oferując wydajność porównywalną z aplikacją uruchamianą natywnie na komputerze. Realizacja części aukcyjnej obejmuje mechanizmy typowe dla serwisów aukcyjnych: publikowanie ofert i zarządzanie licytacjami z odliczaniem czasu do zakończenia, obsługę ofert w czasie rzeczywistym. Interakcja z trójwymiarowym modelem licytowanego przedmiotu będzie możliwa bezpośrednio z poziomu karty aukcji.

2.2 Zakres pracy

Zakres pracy obejmuje następujące kluczowe elementy:

1. Wprowadzenie do technologii WebGPU i WebAssembly.
 - Omówienie podstawowych koncepcji i architektury WebGPU.
 - Opis technologii WebAssembly i jej zastosowań w kontekście aplikacji webowych.
2. Projekt i implementacja komponentu renderującego 3D.
 - Opis projektowania i implementacji komponentu renderującego.
 - Wyjaśnienie procesu kompilacji kodu napisanego w C++ do WebAssembly.
 - Omówienie potencjalnych trudności.
3. Analiza rynku platform aukcyjnych online.
 - Przeprowadzenie analizy głównych platform aukcyjnych.
 - Ocena aktualnie dostępnych technologii wizualizacji.
 - Identyfikacja potrzeb użytkowników i potencjalnych obszarów poprawy.
4. Projekt i implementacja części klienckiej serwisu aukcyjnego.
 - Zaprojektowanie układu stron.

- Stworzenie komponentów frontendowych.
5. Projekt i implementacja części serwerowej serwisu aukcyjnego.
 - Implementacja bazy danych oraz logiki biznesowej.
 - Implementacja ścieżek API.
 6. Integracja z systemem domu aukcyjnego.
 - Opis sposobu włączenia komponentu renderującego do interfejsu użytkownika.
 7. Ewaluacja i perspektywy rozwoju.
 - Testowanie wydajności i testy jednostkowe.
 - Przyszłe zastosowania.
 8. Wnioski i podsumowanie.

2.3 Podział pracy

Komentarz: Gdzie to się ma znajdować?

3 Analiza rynku

Na rynku istnieje wiele serwisów internetowych, na których użytkownicy mogą licytować i kupować przedmioty w czasie rzeczywistym. Jak okazuje się, nawet na największych platformach, nie ma możliwości podglądu obiektu w trzech wymiarach. Witryny takie jak allegro.pl, amazon.pl czy Erli posiadają prostą funkcjonalność podglądu zdjęć, nie wyróżniając się innowacyjnym rozwiązaniem jakim jest rendering 3D licytowanych obiektów. Obrazy 2D zapewniają statyczny widok i nie oddają skomplikowanych szczegółów, kątów lub faktur. Prowadzi to do tego, że nie znamy rzeczywistego stanu przedmiotu, przez co potencjalny konsument, może zostać wprowadzony w błąd, podczas zakupu. Technologia ta pozwala nam na dokładną inspekcję wybranego przez nas przedmiotu, poprzez obracanie obiektem, możliwością przybliżania oraz oddalania się od obiektu i renderingu w bardzo wysokiej jakości.

3.1 allegro.pl

Jest to największy serwis aukcyjny w Polsce, założony w 1999 roku przez Surf Stop Shop. Ponad 33% konsumentów zaczyna właśnie wyszukiwanie produktów na wyżej wymienionym serwisie. Podgląd obiektów na tej stronie jest bardzo prosty, gdyż pozwala nam tylko na podgląd zdjęć, bez możliwości zaawansowanego podglądu licytowanego obiektu. Interfejs użytkownika jest prosty i bardzo intuicyjny, a funkcjonalność licytowania pozwala wielu użytkownikom licytować te same przedmioty bez żadnych problemów. Na tym portalu każdy użytkownik może zostać sprzedawcą i wystawić własne przedmioty, nie korzystając z żadnego pośrednika, co ułatwia kontakt pomiędzy konsumentem a sprzedawcą, dzięki czemu każdy problem dotyczący sprzedaży może zostać rozwiązany w prosty sposób.

3.2 amazon.pl

Amazon to jeden z największych serwisów e-commerce na całym świecie. Startował jako księgrania internetowa, której asortyment systematycznie powiększał się, aż do dzisiejszych rozmiarów. Jediną funkcjonalnością podglądu kupowanego przedmiotu jest przybliżenie obrazka, który zapewnił nam sprzedający. Nie mamy możliwości podglądu realnego stanu licytowanego przedmiotu. Sam proces kupowania czy licytowania jest bardzo intuicyjny i szybki, przez co użytkownik nie ma problemów. Amazon ma bardzo dobrze rozbudowaną obsługę klienta, przez co wszystkie problemy są rozwiązywane szybko i bezproblemowo.

3.3 erli.pl

Założona w 2020 polska firma, która zajmuje się sektorem e-commerce. Zajmuje się sprzedażą przedmiotów z kategorii: meble, budowa i remont, ogród, oświetlenie, narzędzia i wyposażenie domu. Strona ma prosty i przejrzysty układ strony kupowanego przedmiotu, lecz podgląd kupowanego przedmiotu ogranicza się tylko i wyłącznie do powiększenia zdjęć

oferowanych przez sprzedawcę. Takie rozwiązanie może nie oddawać dobrze faktur i tekstur przedmiotu, przez co użytkownik może zostać wprowadzony w skonfundowanie podczas zakupu. Wyświetlane zdjęcia są na małej części strony, co utrudnia ogląd licytowanego przedmiotu dla użytkowników, gdyż muszą powiększać stronę, aby dobrze obejrzeć zdjęcia zawarte przez sprzedającego.

4 Użyte technologie i narzędzia

4.1 Element renderujący

4.1.1 Język C++

Wysokopoziomowy język programowania stworzony przez Bjarne Stroustrupa, jako dodatek do języka C. C++ jest szeroko używany w aplikacjach wymagających wysokiej wydajności, takich jak gry komputerowe, silniki gier komputerowych, aplikacje graficzne, aplikacje uczenia maszynowego i wiele innych, gdzie wydajność jest kluczowa. Perfekcyjnie wpasowują się on w potrzebę stworzenia graficznego komponentu renderującego obiekty 3D.

W przedstawionym projekcie wykorzystywany jest C++ 23 (ISO/IEC 14882:2024), czyli aktualny otwarty standard języka. Ostateczna wersja dokumentu to N4950. Zawiera ona w sobie wiele nowych funkcji, oraz całą zawartość wcześniejszych standardów.

4.1.2 WebGPU

Nowoczesny interfejs programowania aplikacji (API), który umożliwia wydajny dostęp do procesora graficznego (GPU) na różnych platformach. U podstawy działania wykorzystuje systemowe interfejsy: Vulkan, Metal, lub Direct3D 12. Zastępuje starszą technologię WebGL, jako nowy główny standard graficzny dla sieci.

WebGPU jest wspierane zarówno w Google Chrome, jak i Firefox. Mozilla używa własnej implementacji w języku Rust o nazwie wgpu. Google natomiast stworzyło Dawn, czyli implementację standardu WebGPU w Chromium za pomocą C++.

4.1.3 WebGPU-Cpp

Wrapper dla WebGPU napisany w C++, ponieważ obie implementacje udostępniają plik nagłówkowy w języku C z definicjami funkcji i struktur. Stworzony został przez użytkownika eliemichel i udostępniany jest na platformie Github.

4.1.4 WebAssembly

Otwarty standard, który definiuje przenośny format binarny i odpowiadający mu format tekstowy dla programów komputerowych. Głównym celem jest umożliwienie łatwiejszego tworzenia wielce wydajnych aplikacji w przeglądarkach na stronach internetowych. Jest niezależny od platformy oraz wspiera każdy język programowania. Kod wykonywany jest w wirtualnej maszynie stosowej.

4.1.5 Emscripten

Całkowicie otwarty kompilator, który umożliwia kompilację kodu C i C++, lub innego języka używającego LLVM, do formatu WebAssembly. Emcc, czyli frontend kompilatora

używa Clang, oraz LLVM. Pozwala on na bezproblemową konwersję praktycznie każdego projektu C i C++ do WebAssembly. Emscripten ma na koncie kilka udanych konwersji takich jak: Unreal Engine 4, Quake 3, czy Doom 3, wszystkie działające w przeglądarce.

4.1.6 WGSL (WebGPU Shading Language)

Wysokopoziomowy język shaderów dla WebGPU. Umożliwia on tworzenie shaderów, czyli programów wykonywanych na procesorze graficznym. Został stworzony przez W3C GPU for the Web Community Group, aby zapewnić nowoczesny, bezpieczny i przenośny sposób pisania shaderów dla WebGPU.

4.1.7 CMake

Narzędzie do zarządzania budowaniem kodu źródłowego. Zdejmuje z użytkownika konieczność pisania skomplikowanych plików potrzebnych systemom budowania. Jest szeroko używany w projektach C i C++.

4.1.8 Ninja

Lekki system budowania, skupiający się na szybkości. Jest zaprojektowany, aby pliki wejściowe były generowane przez inne wysokopoziomowe narzędzie. Używany do budowania Google Chrome, czy części systemu Android.

4.1.9 Biblioteki C++

- GLFW - Napisana w C wieloplatformowa biblioteka do tworzenia okien aplikacji używających OpenGL, OpenGL ES i Vulkan.
- GLM (OpenGL Mathematics) - Napisana w C++ biblioteka matematyczna przeznaczona dla oprogramowania graficznego opartego na specyfikacjach języka OpenGL Shading Language (GLSL). Biblioteka ta doskonale współpracuje z OpenGL, ale zapewnia kompatybilność z innymi bibliotekami i zestawami SDK innych producentów.
- stb_image - Biblioteka dla C/C++ umożliwiająca załadowywanie obrazów w różnych formatach, takich jak: JPG, PNG, TGA, BMP, PSD, GIF, HDR, PIC.
- tinyobjloader - Napisana w C++ biblioteka umożliwiająca załadowywanie modeli 3D w formacie OBJ stworzonym przez Wavefront Technologies.
- Dear ImGui - Napisana w C++ biblioteka umożliwiająca tworzenie lekkich interfejsów graficznych z minimalnymi zależnościami.

4.2 System aukcyjny



Komentarz: Uzupełnione po implementacji

5 Wprowadzenie do renderowania 3D

Rozpoczynając pracę z zaawansowanymi technikami renderowania 3D w C++, do wyboru są trzy główne biblioteki, które pozwalają na niskopoziomową komunikację z procesorem graficznym: Vulkan, Direct3D 12 oraz Metal. Tylko Vulkan pozwala pisać kod działający na różnych platformach. DirectX jest dostępny wyłącznie na systemie Windows, natomiast Metal jest specyficzny dla ekosystemu Apple. Jednak kod napisany z użyciem tych bibliotek nie może być bezpośrednio uruchomiony w przeglądarce internetowej.

W 2016 roku firma Google zaproponowała stworzenie nowego standardu pozwalającego na komunikację z procesorem graficznym z poziomu przeglądarki internetowej. Miał być to następca WebGL, pozwalający na bardziej efektywne wykorzystanie nowoczesnych kart graficznych. W 2021 roku została opublikowana pierwsza oficjalna specyfikacja pod nazwą WebGPU. Aktualnie na koniec roku 2025 w dalszym ciągu jest to rekomendacja, a nie oficjalny standard W3C. WebGPU jest już wspierane przez większość nowoczesnych przeglądarek internetowych, w tym Google Chrome, Microsoft Edge, Mozilla Firefox oraz Safari. Implementacja WebGPU spaja ze sobą ww. trzy biblioteki, tworząc zunifikowany interfejs umożliwiający pisanie przenośnego kodu renderującego 3D.

Istnieją dwie możliwości korzystania z WebGPU w aplikacjach webowych. Pierwsza z nich to użycie języka programowania JavaScript, które są natywnie obsługiwane przez przeglądarki internetowe. Druga możliwość to skorzystanie z interfejsu w języku C udostępnianego przez dwie istniejące implementacje:

- **Dawn** – implementacja stworzona przez firmę Google, przy pomocy języka C++.
- **wgpu** – implementacja stworzona przez firmę Mozilla, przy pomocy języka Rust.

Następnie napisany kod należy skompilować do formatu WebAssembly. Zaletą tego podejścia jest niemalże porównywalna wydajność aplikacji z wersją natywną, gdyż kod WebAssembly jest uruchamiany bezpośrednio przez silnik przeglądarki, a nie interpretowany jak w przypadku języka JavaScript. Ponadto istniejący już kod można skompilować także do typowego dla systemu operacyjnego formatu wykonywalnego, co pozwala na uruchomienie aplikacji zarówno w przeglądarce internetowej, jak i natywnie na komputerze.

5.1 WebGPU

Najważniejsze w procesie renderowania jest przetworzenie danych przez potok renderujący, przy użyciu procesora graficznego. Komunikacja z procesorem graficznym za pomocą WebGPU odbywa się na dwa sposoby. Pierwszy z nich odbywa się za pomocą obiektów reprezentujących wewnętrzne zasoby GPU. Programista ma niewielki wpływ na ich działanie, jedynie jest w stanie wybrać konkretne, wcześniej zdefiniowane ustawienia, które są zawarte w specjalnych strukturach zwanych deskryptorami. Po zakończeniu działania, zasoby te na-

leży manualnie zwolnić, aby zapobiec wyciekowi pamięci.

zdjeciePrzykladowegoDeskryptora

Drugim sposobem są shadery, czyli programy wykonywane bezpośrednio na procesorze graficznym. Shadery są napisane w specjalnych językach, dla WebGPU jest to WGLSL (WebGPU Shading Language). Niezbędne dla prawidłowego działania potoku renderującego są dwa shadery: vertex shader oraz fragment shader. Vertex shader przetwarza dane wierzchołków, takie jak pozycja, kolor czy współrzędne tekstury. Fragment shader natomiast oblicza kolor każdego piksela na podstawie danych przekazanych z vertex shadera oraz dodatkowych informacji, takich jak oświetlenie czy tekstury.

zdjeciePrzykladowegoShadera

5.2 WebAssembly

W grudniu 2025 roku, 99.24% przeglądarek internetowych wspiera WebAssembly, co czyni go uniwersalnym rozwiązaniem do uruchamiania kodu wymagającego wysokiej wydajności w aplikacjach internetowych. Jednak ten format ten został stworzony jako cel kompilacji, a nie osobny język programowania. W przypadku języka C++ kod źródłowy jest kompilowany do WebAssembly przy pomocy narzędzia Emscripten. Jest to zaawansowane narzędzie, dzięki któremu przeniesiono wiele dużych aplikacji do świata internetowego. Razem z kompilatorem dostarczana także jest specjalna biblioteka, która implementuje standardowe funkcje języków C i C++ w środowisku WebAssembly. Dzięki temu możliwe jest korzystanie z takich funkcji jak alokacja pamięci, operacje na plikach czy obsługa wejścia/wyjścia. Należy jednak pamiętać, że format wyjściowy został stworzony z myślą o bezpieczeństwie i wydajności, dlatego nie wszystkie funkcje języków C i C++ są dostępne lub działają identycznie jak w natywnym środowisku. Maszyna wirtualna, w której wykonywany jest kod nie ma bezpośredniego dostępu do zasobów systemowych, co ogranicza możliwości interakcji z systemem operacyjnym. Emscripten tworzy, więc wirtualny system plików, który symuluje operacje na plikach w pamięci przeglądarki, zachowując przy tym izolację i bezpieczeństwo.

5.3 Podsumowanie

WebGPU w połączeniu z WebAssembly otwiera nowe możliwości dla programistów chcących tworzyć wydajne aplikacje 3D działające bezpośrednio w przeglądarkach internetowych. Dzięki temu możliwe jest tworzenie zaawansowanych wizualizacji, gier oraz interaktywnych aplikacji bez konieczności instalowania dodatkowego oprogramowania. Użycie tych technologii pozwala na pełne wykorzystanie potencjału nowoczesnych kart graficznych, zapewniając jednocześnie szeroką dostępność i łatwość dystrybucji aplikacji.

6 Implementacja komponentu renderującego

Implementację można podzielić w logiczny sposób na trzy główne etapy: inicjalizacja okna, inicjalizacja WebGPU, oraz pętla główna aplikacji. Etapy inicjalizacji muszą być wykonywane w określonej kolejności, dlatego naturalnie układają się w potok. Jednak inaczej niż w typowym potoku zamiast przekazywać między etapami dane, będziemy przekazywać dostarczony przez standard C++ obiekt `std::expected<void, std::string>`, symbolizujący stan wykonania poprzedniego etapu: sukces, lub błąd. Następnie etapy zostaną połączone dzięki operacjom na monadach, znanych z programowania funkcyjnego.

Listing 6.1: Funkcja inicjalizująca komponent renderujący

```
void
initialize()
{
    auto result = init_window()
        .and_then([ this ]() { return init_renderer(); })

    if (!result.has_value())
    {
        std::println(std::cerr, "Error occured:\n{}", result.error());
        std::exit(1);
    }
}
```

6.1 Okno aplikacji

Pierwszym etapem jest utworzenie okna aplikacji, w którym będzie wyświetlany renderowany obraz. W tym celu wykorzystano bibliotekę GLFW, która umożliwia tworzenie okien oraz obsługę zdarzeń wejściowych w sposób niezależny od platformy. Proces inicjalizacji okna obejmuje inicjalizację biblioteki, oraz stworzenie okna. Wskaźnik na utworzone okno jest zapisywany w klasie głównej aplikacji.

Listing 6.2: Część kodu odpowiedzialna za inicjalizację okna

```
auto
init_window() -> std::expected<void, std::string>
{
    if (glfwInit() == GLFW_FALSE)
    {
        return std::unexpected { "Failed to initialize GLFW" };
    }

    glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
    glfwWindowHint(GLFW_RESIZABLE, GLFW_TRUE);
}
```

```

p_window =
    glfwCreateWindow(
        initial_width, initial_height,
        "INZ", nullptr, nullptr);
if (p_window == nullptr)
{
    return std::unexpected { "Failed to create window" };
}
glfwSetWindowUserPointer(p_window, this);

// ...
}

```

Dodatkowo zainicjalizowane zostały funkcje obsługi zdarzeń, takie jak zmiana rozmiaru okna czy obsługa myszy.

Listing 6.3: Dalsza część funkcji inicjalizującej okno - obsługa pozycji myszy

```

auto init_window() -> std::expected<void, std::string>
{
    // ...

    glfwSetCursorPosCallback(
        p_window,
        [](GLFWwindow* window, double x_pos, double y_pos)
        {
            if (
                auto* app =
                    reinterpret_cast<application*>(
                        glfwGetWindowUserPointer(window));
                app != nullptr)
            {
                app->cursor_position_callback(x_pos, y_pos);
            }
        });

    // ...
}

```

6.2 WebGPU

Inicjalizacja WebGPU składa się z trzynastu kroków, które muszą być wykonane w określonej kolejności, aby poprawnie skonfigurować środowisko renderujące. Poniżej przedstawiono poszczególne kroki wraz z opisem ich funkcji.

6.2.1 Adapter

Adapter to reprezentacja fizycznego lub wirtualnego urządzenia GPU dostępnego w systemie. W tym kroku aplikacja wysyła zapytanie do systemu o dostępne adaptory, a następnie wybiera najbardziej odpowiedni na podstawie określonych kryteriów, takich jak wydajność czy obsługiwane funkcje.

6.2.2 Device

Device to logiczne przedstawienie wybranego adaptera, które umożliwia tworzenie zasobów procesora graficznego oraz wykonywanie operacji renderujących. W tym kroku aplikacja tworzy obiekt Device, który będzie używany do komunikacji z procesorem graficznym.

6.2.3 Command queue

Skoro Device jest już utworzony, należy teraz stworzyć kolejkę poleceń (Command queue). Dzięki niej kod aplikacji wykonywany na CPU może wysyłać polecenia do GPU w sposób asynchroniczny. W tym kroku aplikacja tworzy obiekt Command queue.

6.2.4 Shader module

Podczas tego kroku aplikacja ładuje i kompiluje shadery, które będą używane w trakcie potoku renderującego. Shadery są napisane w języku WGSL i definiują sposób przetwarzania wierzchołków oraz obliczania koloru pikseli.

6.2.5 Depth texture

Jest to specjalny obiekt do przechowywania informacji o głębi sceny 3D. W tym kroku aplikacja tworzy teksturę głębi, która będzie używana do testów głębi podczas potoku renderującego.

6.2.6 Bind group layout

Bind group layout definiuje interfejs między zbiorem zasobów powiązanych w bind group (tworzony na końcu) a ich dostępnością w etapach shadera. Definiowane są tutaj typy zasobów, ich widoczność w poszczególnych shaderach.

6.2.7 Pipeline

Głównym elementem procesu renderującego jest potok renderujący (Render Pipeline). W tym kroku aplikacja tworzy obiekt Pipeline, który definiuje wszystkie etapy przetwarzania danych. Korzystając z oficjalnego standardu WebGPU, można wyszczególnić następujące etapy potoku renderującego:

1. Vertex fetch
2. Vertex shader
3. Primitive assembly
4. Rasterization
5. Fragment shader
6. Stencil test and operation
7. Depth test and write
8. Output merging

6.2.8 Sampler

6.2.9 Textures

6.2.10 Geometry

6.2.11 Uniforms

6.2.12 Lighting uniforms

6.2.13 Bind group

6.3 Pętla główna